

UNIVERSIDAD DEL VALLE DE GUATEMALA
REDES- SECCIÓN 10

Catedrático: Kevin Antonio Velázquez Aguilar



Proyecto 1 entrega 1

23661 – Mérida Jiménez, Ángel de Jesús

Excelencia que trasciende

Nueva Guatemala de la Asunción, 20 de agosto de 2026

¿Qué se construyó?

En esta primera entrega se implementó un servidor MCP local para una cadena de farmacias, comunicandose por JSON-RPC 2.0 implementado a mano, sin FastMCP ni ningún SDK de MCP. El servidor expone cinco tools: `search_medications`, `check_inventory`, `suggest_products_for_symptom`, `create_order` y `get_order_status`, respaldadas por un catálogo, inventario y sistema de pedidos ficticios.

Conclusiones:

Implementar JSON-RPC manualmente ayudó a entender que MCP no es un protocolo complejo en sí mismo, sino una capa delgada de convenciones (el ciclo `initialize / notifications/initialized / tools/list / tools/call`) construida sobre un formato de mensajes ya existente. Ver esa separación en el código -la capa de JSON-RPC no sabe nada de MCP, y la capa de MCP no sabe nada del transporte- deja claro porque el protocolo pudo convertirse en un estándar adoptado por distintas empresas: cada pieza puede implementarse y probarse por separado.

Separar el proyecto en transporte, núcleo de protocolo y tools también resultó útil en la práctica: cuando hubo que reemplazar una tool de prueba (`echo`) por las tools reales de farmacia, no fue necesario tocar ni el transporte ni el manejo de `tools/call`, solo el registro de tools. Esa misma separación es la que va a permitir, en la segunda entrega, sustituir el transporte por `stdio` por uno HTTP sin reescribir la lógica del protocolo.

Un aprendizaje concreto fue que `stdout` es un canal de protocolo, no un canal de depuración: cualquier texto que no sea un mensaje JSON-RPC válido rompe el stream. Por eso todo el logging del servidor se dirige a `stderr` y a un archivo de log, nunca a la salida estándar.

En el diseño de las tools se optó por que el servidor exponga datos sin tomar decisiones por el usuario. Por ejemplo, `check_inventory` reporta si un producto requiere receta, pero no bloquea la consulta; en cambio, `create_order` si rechaza productos con receta, porque ahí ya se trata de una acción (una venta), no de información. Separar "mostrar información" de "tomar una acción" pareció la forma más honesta de diseñar un servicio de datos que después será usado por un LLM.

La tool de síntomas fue la más delicada de diseñar. Se decidió que, ante síntomas de alarma (dolor en el pecho, dificultad para respirar, embarazo, síntomas en un bebe, entre otros), el servidor no sugiera ningún producto y en su lugar remita a atención médica. La razón es que un chatbot de farmacia que "recomienda" un analgésico ante una posible emergencia sería un mal diseño, sin importar que tan bien implementado este el protocolo por debajo.

Dificultades

Al agregar las tools reales, el archivo de pruebas del handshake (`test_handshake.py`) seguía haciendo referencia a la tool de prueba (`echo`) que ya se había eliminado del registro, lo que hacía fallar una de las verificaciones, aunque el servidor funcionara bien. Sirvió para notar que las pruebas necesitan actualizarse en el mismo commit que el código que prueban, no después.

Lecciones aprendidas

Con el servidor terminado, quedo más claro porque, el enunciado exige implementar JSON-RPC a mano en vez de usar un SDK: usar FastMCP hubiera ocultado precisamente las partes que mas ayudaron a entender el protocolo (el manejo de notificaciones sin respuesta, los códigos de error estándar, la separación entre errores de protocolo y errores de ejecución de una tool).

Pendiente para la segunda entrega

- Chatbot anfitrión: conexión con la API del LLM, contexto de sesión y log de interacciones con los servidores MCP (funcionalidades 1-3).
- Conexión con los servidores oficiales de Filesystem y Git (funcionalidad 4).
- Despliegue del mismo servidor de farmacia de forma remota (funcionalidad 6).
- Captura y análisis con Wireshark del tráfico entre el anfitrión y el servidor remoto (funcionalidades 7 y 9).